

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÀI TẬP LỚN

CÁC HỆ THỐNG PHÂN TÁN

Xây dựng hệ thống Thương mại điện tử C2C
sử dụng kiến trúc Microservices

Các sinh viên thực hiện:

Nguyễn Thành Công

Tạ Kim Ngân

Bùi Thái Sỹ

Giảng viên giảng dạy:

Thầy Kim Ngọc Bách

HÀ NỘI- NĂM 2026

MỤC LỤC

MỤC LỤC	
DANH MỤC HÌNH ẢNH.....	
DANH MỤC BẢNG BIỂU	
THUẬT NGỮ VIẾT TẮT	
LỜI CẢM ƠN.....	
LỜI MỞ ĐẦU	
CHƯƠNG 1. Tổng quan và mô tả bài toán	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu vào xây dựng	2
1.3 Đối tượng và phạm vi.....	2
1.4 Yêu cầu hệ thống	3
1.5 Phương pháp nghiên cứu	3
1.6 Bố cục báo cáo.....	4
CHƯƠNG 2. KIẾN TRÚC THỐNG.....	5
2.1 Sơ đồ kiến trúc triển khai tại local.....	5
2.2 Sơ đồ kiến trúc triển khai trên GCP (Google Cloud Platform)	8
CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE	11
3.1 Các microservice hoạt động	11
3.2 Xử lý khi service tạm thời không phản hồi	13
CHƯƠNG 4. THIẾT KẾ RESTFUL API	15
4.1 Nguyên tắc thiết kế api chung	15
4.2 Kiến trúc phân hệ API	15
CHƯƠNG 5. Luồng message queue (Redis Streams).....	18
5.1 Cơ sở lý thuyết về Redis Streams và Consumer Group	18
5.2 Kiến trúc Cross-service Cache bằng Streams.....	18
5.2.1 Cấu trúc bộ nhớ đệm đa tầng.....	19
5.2.2 Luồng hoạt động theo mô hình hướng sự kiện.....	19
5.3 Chi tiết các phân luồng và cấu trúc payload.....	19
5.3.1 Stream đồng bộ cache cửa hàng	19
5.3.2 Stream đồng bộ cache sản phẩm.....	20
5.3.3 Stream sự kiện đơn hàng	20
5.4 Cơ chế chịu lỗi và Dead Letter Queue.....	21

5.5 Đánh giá lựa chọn Redis Streams so với Pub/Sub	21
CHƯƠNG 6. Thử nghiệm và đánh giá.....	23
6.1 Môi trường triển khai và thử nghiệm.....	23
6.2 Kịch bản kiểm thử	24
6.2.1 Kiểm thử chức năng nghiệp vụ.....	24
6.2.2 Kiểm thử chức năng chịu lỗi và tính bền bỉ	28
6.3 Đánh giá ưu điểm của hệ thống	29
6.4 Hạn chế và hướng phát triển trong tương lai.....	30
TÀI LIỆU THAM KHẢO	32
PHỤ LỤC	33
PHỤ LỤC A: ĐẶC TẢ CHI TIẾT CÁC ENDPOINT API.	34

DANH MỤC HÌNH ẢNH

Hình 2.1 Sơ đồ kiến trúc hệ thống triển khai tại local.....	5
Hình 2.2 Sơ đồ kiến trúc triển khai trên GCP.....	8
Hình 6.1: Giao diện trang chủ	24
Hình 6.2: Giao diện đăng nhập.....	25
Hình 6.3: Giao diện quản lý sản phẩm của shop	26
Hình 6.4: Giao diện thanh toán.....	26
Hình 6.5: Giao diện thanh toán điện tử với VNPay	27
Hình 6.6: Giao diện chat realtime.....	28
Hình 6.7: Giao diện tìm kiếm sản phẩm.....	28

DANH MỤC BẢNG BIỂU

Bảng 4.1 Danh mục các vi dịch vụ và nhóm nghiệp vụ cốt lõi	15
Bảng C.1: Đặc tả API của Gateway Service	34
Bảng C.2: Đặc tả API của Auth Service.....	34
Bảng C.3: Đặc tả API của User Service	35
Bảng C.4: Đặc tả API của Shop Service	36
Bảng C.5: Đặc tả API của Product Service	37
Bảng C.6: Đặc tả API của Order Service	38
Bảng C.7: Đặc tả API của Payment Service.....	39
Bảng C.8: Đặc tả API của Chat Service	39
Bảng C.9: Đặc tả API của Upload Service.....	40

THUẬT NGỮ VIẾT TẮT

Thuật ngữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
ACID	Atomicity, Consistency, Isolation, Durability	Nguyên tắc: Tính nguyên tử, Nhất quán, Cô lập, Bền vững trong cơ sở dữ liệu
API	Application Programming Interface	Giao diện lập trình ứng dụng
C2C	Customer-to-Customer	Mô hình thương mại điện tử từ người tiêu dùng đến người tiêu dùng
CDC	Change Data Capture	Kỹ thuật bắt luồng thay đổi dữ liệu
CI	Continuous Integration	Tích hợp liên tục
CRUD	Create, Read, Update, Delete	Các thao tác cơ bản trong cơ sở dữ liệu: Tạo, Đọc, Cập nhật, Xóa
DLQ	Dead Letter Queue	Hàng đợi thư chết / Hàng đợi xử lý thông điệp lỗi
DNS	Domain Name System	Hệ thống phân giải tên miền
ELK	Elasticsearch, Logstash, Kibana	Bộ công cụ quản lý, thu thập và phân tích log hệ thống
GCP	Google Cloud Platform	Nền tảng điện toán đám mây của Google
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Ký pháp đối tượng JavaScript (Định dạng trao đổi dữ liệu)
JWT	JSON Web Token	Chuỗi mã thông báo xác thực an toàn định dạng JSON
REST	Representational State Transfer	Trạng thái chuyển giao đại diện (Kiến trúc phần mềm)

SKU	Stock Keeping Unit	Đơn vị lưu kho / Biến thể sản phẩm
UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
URL	Uniform Resource Locator	Trình định vị tài nguyên thống nhất / Đường dẫn web
VM	Virtual Machine	Máy ảo

LỜI CẢM ƠN

Để hoàn thành bài tập lớn môn học "Các hệ thống phân tán" với đề tài “Xây dựng hệ thống Thương mại điện tử C2C sử dụng kiến trúc Microservices”, nhóm chúng em đã nhận được sự hướng dẫn, chỉ bảo và động viên nhiệt tình từ nhiều cá nhân và tập thể.

Trước hết, chúng em xin trân trọng gửi lời cảm ơn sâu sắc nhất tới thầy Kim Ngọc Bách, giảng viên phụ trách môn học. Thầy đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn cốt lõi về kiến trúc phân tán và định hướng nghiên cứu cho nhóm. Những lời góp ý, giải đáp thắc mắc của thầy trong suốt quá trình thực hiện bài tập lớn chính là cơ sở quan trọng giúp chúng em vượt qua các thách thức kỹ thuật và hoàn thiện hệ thống này. Sự nhiệt huyết và phương pháp truyền đạt khoa học của thầy là nguồn động lực rất lớn đối với chúng em.

Chúng em cũng xin gửi lời cảm ơn chân thành đến Ban Giám đốc Học viện Công nghệ Bru chính Viễn thông, cùng toàn thể quý thầy cô trong Khoa Công nghệ Thông tin đã tạo ra một môi trường học tập, nghiên cứu năng động và trang bị cho chúng em nền tảng kiến thức vững chắc trong những năm học vừa qua.

Mặc dù đã nỗ lực hết mình trong việc tìm hiểu công nghệ và phát triển hệ thống, nhưng do kiến thức chuyên môn và kinh nghiệm triển khai thực tế các hệ thống phân tán quy mô lớn còn nhiều hạn chế, bài báo cáo chắc chắn không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự châm chước và những ý kiến đóng góp quý báu từ thầy để đề tài được hoàn thiện hơn, đồng thời giúp chúng em rút ra những bài học kinh nghiệm sâu sắc cho công việc sau này.

Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Nguyễn Thành Công

Tạ Kim Ngân

Bùi Thái Sỹ

LỜI MỞ ĐẦU

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang lan tỏa mạnh mẽ trên toàn cầu, công nghệ Internet vạn vật và Trí tuệ nhân tạo đã và đang khẳng định vai trò là những công nghệ nền tảng, tạo ra những bước đột phá trong nhiều lĩnh vực kinh tế - xã hội, đặc biệt là nông nghiệp. Nông nghiệp thông minh, hay nông nghiệp chính xác, không còn là một khái niệm xa vời mà đã trở thành một xu hướng tất yếu, giúp tối ưu hóa quy trình sản xuất, nâng cao năng suất, giảm thiểu chi phí và ứng phó hiệu quả với những thách thức từ biến đổi khí hậu.

Trong kỷ nguyên số hóa bùng nổ hiện nay, thương mại điện tử đã và đang khẳng định vai trò là một trong những trụ cột quan trọng nhất của nền kinh tế toàn cầu. Việc mua sắm trực tuyến không còn là một khái niệm mới mẻ mà đã trở thành một xu hướng tất yếu, tạo ra những bước đột phá trong hành vi tiêu dùng và luân chuyển hàng hóa. Để đáp ứng nhu cầu này, các hệ thống phần mềm nền tảng đòi hỏi phải liên tục tiến hóa, tối ưu hóa quy trình xử lý, nâng cao năng suất hệ thống và ứng phó hiệu quả với những thách thức từ lượng dữ liệu khổng lồ sinh ra mỗi ngày.

Các nền tảng thương mại điện tử hiện đại, đặc biệt là mô hình C2C, cho phép một người dùng có thể linh hoạt chuyển đổi giữa vai trò người mua và người bán, tạo ra một mạng lưới tương tác vô cùng phức tạp. Tuy nhiên, nhiều hệ thống hiện nay vẫn được xây dựng dựa trên kiến trúc nguyên khối truyền thống. Cấu trúc này bộc lộ rõ những hạn chế khi lượng truy cập tăng đột biến, gây khó khăn cho việc mở rộng hệ thống một cách linh hoạt. Việc xử lý tập trung mọi nghiệp vụ vào một khối duy nhất không chỉ làm lãng phí tài nguyên mà còn tiềm ẩn nguy cơ hiệu ứng domino, nơi một lỗi rò rỉ bộ nhớ nhỏ ở một chức năng cũng có thể kéo theo sự sụp đổ của toàn bộ nền tảng giao dịch.

Trước thực trạng đó, việc ứng dụng các tư tưởng thiết kế hiện đại để phân rã và thông minh hóa nền tảng thương mại điện tử trở nên vô cùng cấp thiết. Hệ thống phân tán, nổi bật với kiến trúc dịch vụ vi mô, cung cấp một giải pháp toàn diện. Kiến trúc này cho phép chia nhỏ hệ thống thành các dịch vụ độc lập, quản lý cơ sở dữ liệu riêng biệt và giao tiếp với nhau qua các luồng sự kiện. Hệ thống phân tán có khả năng tự động xử lý lỗi cục bộ, đảm bảo tính sẵn sàng cao và duy trì sự nhất quán của dữ liệu giao dịch trên diện rộng.

Xuất phát từ những yêu cầu thực tiễn đó, kết hợp với các kiến thức nền tảng của môn học, nhóm chúng em đã thực hiện đề tài: “Xây dựng hệ thống Thương mại điện tử SunStack theo kiến trúc Microservices”. Bài tập lớn này hướng tới việc thiết kế và triển khai một nền tảng E-Commerce phân tán hoàn chỉnh, chia nhỏ thành các module độc lập như định danh, quản lý sản phẩm, đặt hàng và thanh toán. Hệ thống tích hợp các công nghệ quản lý luồng sự kiện thời gian thực và được đóng gói toàn diện trên môi trường Container. Giải pháp này không chỉ giúp quản trị hệ thống dễ dàng cấu hình, mở rộng theo tải thực tế mà còn đảm bảo tính chịu lỗi, qua đó góp phần nâng cao hiệu năng và trải nghiệm mua sắm xuyên suốt của người dùng.

Bài báo cáo này sẽ trình bày chi tiết các nội dung nghiên cứu của bài tập lớn, từ tổng quan và cơ sở lý thuyết, đến quá trình thiết kế kiến trúc, triển khai luồng nghiệp vụ, thử nghiệm và đánh giá hệ thống.

CHƯƠNG 1. TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số hiện nay, thương mại điện tử đã trở thành trụ cột quan trọng của nền kinh tế toàn cầu nói chung và Việt Nam nói riêng. Sự dịch chuyển thói quen mua sắm của người tiêu dùng đã thúc đẩy các mô hình kinh doanh trực tuyến phát triển mạnh mẽ. Trong đó, mô hình thương mại điện tử C2C vươn lên như một xu hướng tất yếu, tiêu biểu qua các nền tảng như Shopee, Lazada hay Taobao. Đặc trưng của mô hình này là tính linh hoạt cao: một người dùng vừa có thể đóng vai trò là người mua hàng, vừa có thể dễ dàng khởi tạo gian hàng để trở thành người bán. Hệ sinh thái hai chiều này tạo ra một lượng dữ liệu và giao dịch khổng lồ, diễn ra liên tục hàng ngày.

Sự phát triển bùng nổ này đặt ra những bài toán hóc búa về mặt kỹ thuật cho quá trình xây dựng hệ thống. Các nền tảng E-Commerce hiện đại phải đối mặt với các đợt tăng trưởng lưu lượng truy cập đột biến trong các chiến dịch khuyến mãi (Flash Sale, Siêu sale ngày đôi). Nếu sử dụng kiến trúc nguyên khối truyền thống, hệ thống sẽ bộc lộ nhiều hạn chế chí mạng:

- Khả năng mở rộng kém: Khi một tính năng cụ thể (ví dụ: Thanh toán hoặc Đặt hàng) bị quá tải, toàn bộ khối ứng dụng phải được nhân bản, gây lãng phí tài nguyên máy chủ.
- Một điểm chịu lỗi duy nhất: Một lỗi rò rỉ bộ nhớ ở module xử lý hình ảnh cũng có thể làm sập toàn bộ hệ thống mua sắm.
- Khó khăn trong bảo trì và triển khai: Mã nguồn phình to khiến thời gian build và deploy kéo dài, gây cản trở tốc độ phát hành tính năng mới.

Nhận thức được những thách thức đó, việc chuyển đổi sang kiến trúc dịch vụ vi mô là giải pháp tối ưu được các ông lớn công nghệ áp dụng. Kiến trúc này phân rã một hệ thống lớn thành các dịch vụ nhỏ gọn, độc lập, có cơ sở dữ liệu riêng và giao tiếp với nhau qua mạng.

Xuất phát từ nhu cầu thực tiễn và mong muốn làm chủ các công nghệ thiết kế hệ thống phân tán hiện đại, đề tài "Xây dựng hệ thống Thương mại điện tử SunStack theo

kiến trúc Microservices" đã được lựa chọn nghiên cứu và thực hiện. Đề tài không chỉ giải quyết bài toán nghiệp vụ cốt lõi của một sàn C2C mà còn đi sâu vào việc xử lý các vấn đề giao tiếp liên dịch vụ và tính nhất quán dữ liệu trong môi trường phân tán.

1.2 Mục tiêu nghiên cứu vào xây dựng

Đề án được thực hiện nhằm giải quyết hai nhóm mục tiêu chính:

Mục tiêu lý thuyết:

- Nghiên cứu kiến trúc Microservices: Ưu, nhược điểm và các pattern thiết kế phổ biến (API Gateway, Database per service, Event-driven architecture).
- Tìm hiểu cơ chế giao tiếp bất đồng bộ giữa các dịch vụ thông qua Message Broker, cụ thể là ứng dụng Redis Streams.
- Nghiên cứu giải pháp đảm bảo tính nhất quán dữ liệu cuối trong môi trường phân tán.

Mục tiêu thực tiễn:

- Thiết kế và phát triển thành công nền tảng E-Commerce C2C SunStack.
- Đóng gói và triển khai hệ thống thông qua các công nghệ Containerization.
- Xây dựng hoàn chỉnh luồng nghiệp vụ từ khâu xác thực, quản lý gian hàng, sản phẩm, đến đặt hàng và thanh toán trực tuyến.
- Triển khai hệ thống ELK để tập trung log từ các service.
- Triển khai hệ thống trên Google Cloud Platform.

1.3 Đối tượng và phạm vi

Đối tượng nghiên cứu:

- Các công nghệ Backend Backend: Python, cơ sở dữ liệu phi cấu trúc MongoDB.
- Hệ thống bộ nhớ đệm và Message Broker: Redis Cache, Redis Streams.
- Luồng quy trình nghiệp vụ E-Commerce mô hình C2C.
- Log từ các service.

Phạm vi hệ thống:

- Phạm vi chức năng: Tập trung mô phỏng luồng nghiệp vụ lõi giữa khách hàng và người bán. Hệ thống xử lý các tác vụ quản lý người dùng, sản phẩm, giỏ hàng, đơn hàng, thanh toán, và giao tiếp thời gian thực.
- Phạm vi kỹ thuật: Hệ thống Backend được chia thành 9 microservices độc lập bao gồm: Gateway, Auth, User, Shop, Product, Order, Payment, Chat, Upload. Front-end được xây dựng ở mức độ minh họa UI/UX để tương tác với các API dịch vụ.

1.4 Yêu cầu hệ thống

Hệ thống được thiết kế hướng tới việc đáp ứng các tiêu chuẩn khắt khe của một kiến trúc phân tán:

- Tính sẵn sàng cao và khả năng chịu lỗi: Hệ thống phải duy trì hoạt động ngay cả khi một vài node dịch vụ gặp sự cố.
 - o Áp dụng các cơ chế Fallback, Circuit Breaker và Retry tại tầng API Gateway trung tâm để ngăn chặn lỗi dây chuyền.
- Tính đóng gói dữ liệu: Tuân thủ nguyên tắc mỗi Microservice sở hữu và quản lý một database độc lập.
 - o Tuyệt đối không truy xuất chéo trực tiếp vào cơ sở dữ liệu của dịch vụ khác để tránh hiện tượng dính líu chặt.
- Tính nhất quán dữ liệu cuối: Chấp nhận độ trễ nhỏ trong việc đồng bộ dữ liệu để đổi lấy hiệu năng cao.
 - o Sử dụng Redis Streams làm xương sống để luân chuyển các sự kiện thay vì gọi API đồng bộ cho các tác vụ cập nhật chéo, đảm bảo dữ liệu toàn hệ thống cuối cùng sẽ được đồng nhất một cách chính xác.
- Khả năng mở rộng: Cho phép scale up hoặc scale out cục bộ cho các service có tải cao như Product, Order mà không ảnh hưởng đến toàn hệ thống.

1.5 Phương pháp nghiên cứu

Để đạt được các mục tiêu đã đề ra, đồ án sử dụng các phương pháp tiếp cận sau:

- Phương pháp nghiên cứu tài liệu: Thu thập, phân tích tài liệu kỹ thuật từ các nguồn chính thống về Microservices, Redis, MongoDB và thiết kế hệ thống E-commerce.
- Phương pháp phân tích và thiết kế: Mô hình hóa các yêu cầu bài toán thành sơ đồ kiến trúc hệ thống, sơ đồ tuần tự và sơ đồ quan hệ dữ liệu cho từng service.
- Phương pháp thực nghiệm: Lập trình xây dựng ứng dụng, kiểm thử API liên dịch vụ và đánh giá luồng dữ liệu thông qua các công cụ như Postman, Docker.

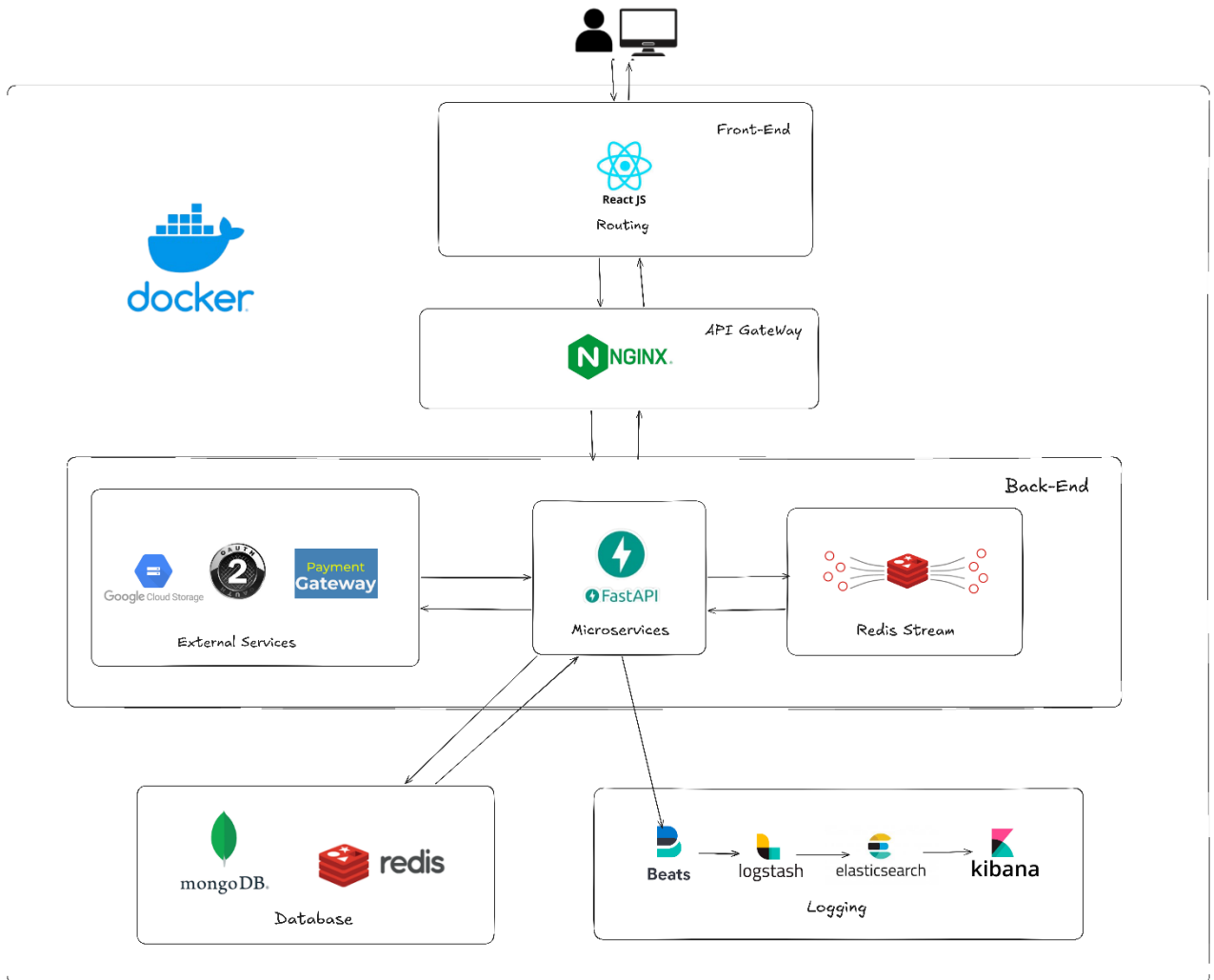
1.6 Bố cục báo cáo

Nội dung của báo cáo được chia thành 6 chương như sau:

- Chương 1: Tổng quan và mô tả bài toán
- Chương 2: Kiến trúc hệ thống
- Chương 3: Mô tả các Microservices
- Chương 4: Thiết kế RESTful API
- Chương 5: Luồng message queue (Redis Stream)
- Chương 6: Thử nghiệm và đánh giá

CHƯƠNG 2. KIẾN TRÚC THÔNG

2.1 Sơ đồ kiến trúc triển khai tại local



Hình 2.1 Sơ đồ kiến trúc hệ thống triển khai tại local

Hệ thống SunStack được xây dựng theo kiến trúc microservices, trong đó các thành phần nghiệp vụ được tách biệt thành các service độc lập, giao tiếp với nhau thông qua các cơ chế được định nghĩa rõ ràng. Toàn bộ hệ thống được đóng gói và triển khai bằng Docker, đảm bảo tính nhất quán giữa môi trường phát triển và vận hành. Kiến trúc tổng thể được chia thành bốn tầng chính: tầng Front-End, tầng API Gateway, tầng xử lý Back-End, và tầng Data và Logging.

Giao diện người dùng được xây dựng bằng ReactJS với cơ chế định tuyến phía client. Người dùng tương tác với hệ thống thông qua trình duyệt web; mọi yêu cầu từ phía client đều được chuyển tiếp xuống tầng API Gateway thay vì gọi trực tiếp vào các

service nghiệp vụ, đảm bảo sự tách biệt và kiểm soát luồng truy cập từ một điểm duy nhất.

Đứng giữa Front-End và Back-End là NGINX, đóng vai trò API Gateway — điểm vào duy nhất của toàn bộ hệ thống backend. NGINX đảm nhận các chức năng như reverse proxy, định tuyến request đến đúng microservice phía sau, và SSL termination. Việc tập trung luồng vào qua Gateway giúp hệ thống dễ dàng kiểm soát xác thực, giới hạn tốc độ và theo dõi traffic mà không cần can thiệp vào từng service riêng lẻ.

Tầng Back-End bao gồm ba nhóm thành phần chính hoạt động phối hợp với nhau:

Nhóm Microservices được xây dựng trên nền FastAPI, gồm các service tách theo domain nghiệp vụ: gateway, auth-service, user-service, product-service, order-service, shop-service, payment-service, chat-service và upload-service. Mỗi service chạy độc lập, có cổng riêng và chỉ giao tiếp với nhau thông qua HTTP nội bộ hoặc qua Redis Streams, nhờ đó giảm thiểu sự phụ thuộc trực tiếp giữa các service.

Nhóm External Services bao gồm các dịch vụ bên ngoài mà hệ thống tích hợp: Google Cloud Storage để lưu trữ tệp và hình ảnh, OAuth2 để xác thực qua tài khoản Google, và Payment Gateway (VNPay) để xử lý thanh toán trực tuyến. Các microservice giao tiếp hai chiều với nhóm này tùy theo nghiệp vụ: ví dụ upload-service đẩy file lên GCS, payment-service gọi VNPAY để tạo URL thanh toán và nhận callback kết quả.

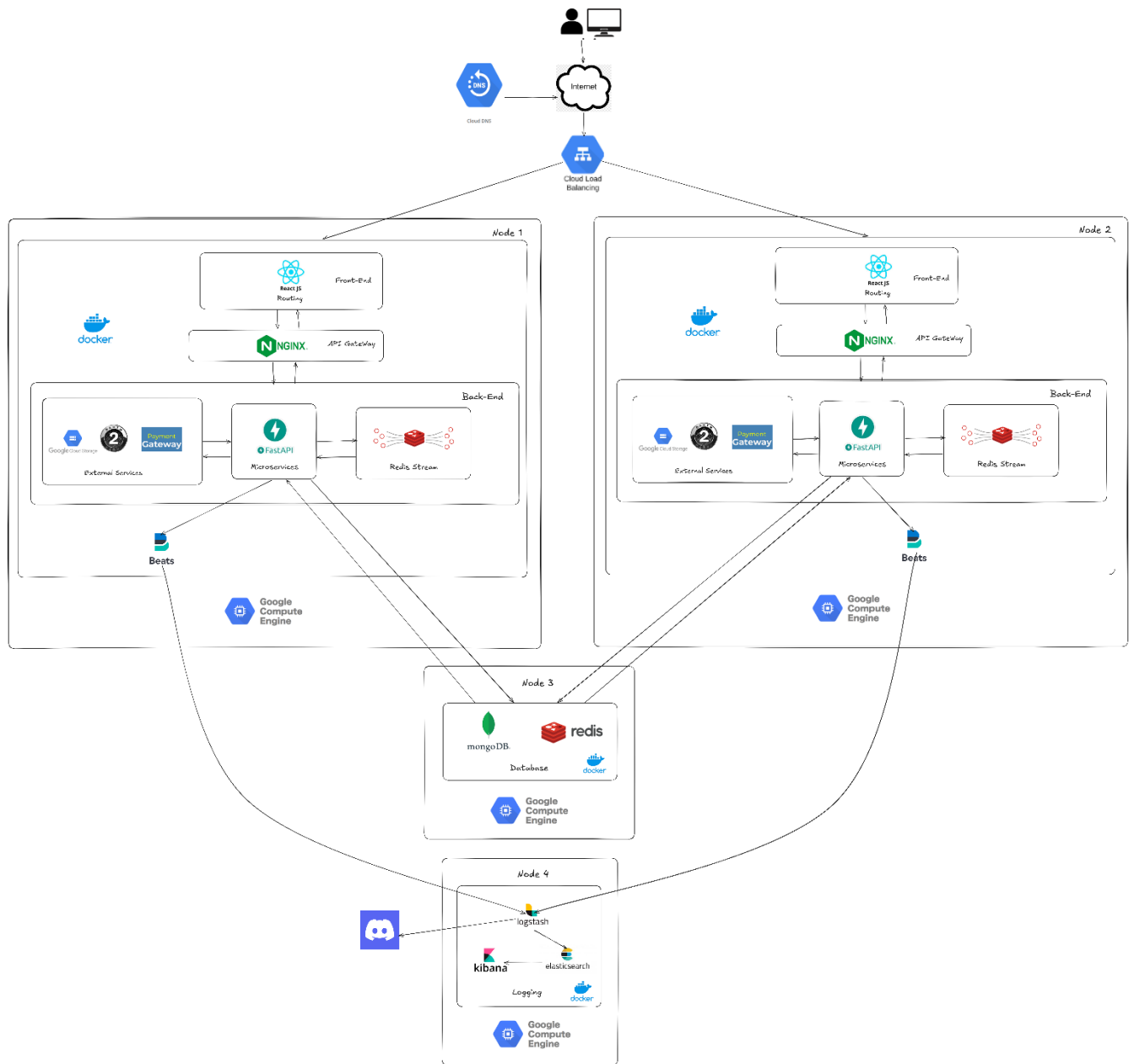
Redis Stream được đặt riêng trong tầng Back-End, đóng vai trò là message broker nội bộ theo mô hình event-driven. Các microservice publish sự kiện (như tạo đơn hàng, thay đổi sản phẩm) vào các stream tương ứng; các service tiêu thụ (consumer) đọc message theo consumer group, xử lý và ghi nhận sau khi hoàn tất. Cơ chế này cho phép các service đồng bộ cache cho nhau mà không cần gọi HTTP trực tiếp — ví dụ product-service và order-service có thể đọc thông tin shop từ cache cục bộ được đồng bộ qua stream service:shop:updates, thay vì phải gọi sang shop-service mỗi request. Trong trường hợp xử lý message thất bại, hệ thống có cơ chế retry và chuyển message lỗi sang DLQ để tránh "poison message" làm kẹt toàn bộ consumer.

Hệ thống sử dụng song song hai công nghệ lưu trữ. MongoDB là cơ sở dữ liệu chính, mỗi microservice sử dụng một database riêng biệt (ví dụ auth_db, order_db,

shop_db...) nhằm đảm bảo tính độc lập về dữ liệu giữa các service — đây là nguyên tắc cơ bản của microservices. Redis phục vụ hai mục đích song song: làm bộ nhớ đệm để giảm tải truy vấn MongoDB cho các dữ liệu truy cập thường xuyên, và làm hạ tầng cho Redis Streams như đã mô tả ở trên.

Hệ thống tích hợp ELK Stack để thu thập, xử lý và trực quan hóa log. Filebeat được triển khai trên các node ứng dụng, đọc log từ thư mục ./logs và chuyển tiếp về Logstash để parse và phân loại. Sau đó log được lưu vào Elasticsearch để phục vụ truy vấn, và hiển thị trực quan qua Kibana — cho phép đội vận hành theo dõi trạng thái hệ thống, phát hiện lỗi và tạo dashboard quan sát theo thời gian thực.

2.2 Sơ đồ kiến trúc triển khai trên GCP (Google Cloud Platform)



Hình 2.2 Sơ đồ kiến trúc triển khai trên GCP

Ở môi trường production, hệ thống SunStack được triển khai lên Google Cloud Platform với mô hình đa node, sử dụng Google Compute Engine làm nền tảng máy ảo. Thay vì chạy tập trung trên một máy như môi trường local, toàn bộ hệ thống được phân tách thành bốn node độc lập theo chức năng, kết nối với nhau qua mạng nội bộ của GCP. Mô hình này hướng tới khả năng chịu tải cao hơn, dễ mở rộng theo chiều ngang và tách biệt rõ ràng giữa các tầng ứng dụng, dữ liệu và quan sát.

Luồng truy cập từ người dùng Internet đi qua hai lớp trước khi chạm đến ứng dụng. Đầu tiên, Cloud DNS của GCP phân giải tên miền và định tuyến traffic về địa chỉ IP public của hệ thống. Tiếp theo, Cloud Load Balancing đảm nhận vai trò cân bằng tải, phân phối request đến các node ứng dụng (Node 1 và Node 2) theo thuật toán cân bằng tải. Lớp này đảm bảo hệ thống không có điểm lỗi đơn ở tầng vào, đồng thời cho phép mở rộng số lượng node ứng dụng mà không cần thay đổi cấu hình DNS hay phía client.

Hai node ứng dụng có cấu trúc đối xứng hoàn toàn, mỗi node là một Google Compute Engine VM chạy toàn bộ stack ứng dụng được đóng gói bằng Docker. Bên trong mỗi node, kiến trúc được tổ chức theo ba lớp giống với môi trường local: React JS phục vụ giao diện người dùng với client-side routing, NGINX đóng vai trò API Gateway tiếp nhận và định tuyến request, và tầng Back-End gồm các FastAPI Microservices xử lý nghiệp vụ, kết nối với External Services (Google Cloud Storage, OAuth2, Payment Gateway) và trao đổi sự kiện nội bộ qua Redis Stream cục bộ của từng node.

Việc triển khai stack đầy đủ trên mỗi node theo mô hình này giúp các node hoạt động độc lập — nếu một node gặp sự cố, Load Balancer có thể chuyển toàn bộ traffic sang node còn lại mà không gián đoạn dịch vụ. Mỗi node cũng có một Filebeat chạy độc lập bên ngoài stack Docker, có nhiệm vụ đọc log được ghi ra từ các container và chuyển tiếp về Node 4 để xử lý tập trung.

Toàn bộ dữ liệu được tập trung tại Node 3, một Google Compute Engine VM chuyên biệt chạy MongoDB và Redis trong Docker. Việc tách tầng dữ liệu ra một node riêng mang lại một số lợi ích quan trọng: Node 1 và Node 2 có thể được scale ngang mà không ảnh hưởng đến dữ liệu; tài nguyên phần cứng (CPU, RAM, disk I/O) cho MongoDB và Redis có thể được cấu hình độc lập theo nhu cầu; và việc backup, snapshot dữ liệu được thực hiện tập trung tại một điểm duy nhất.

Cả hai node ứng dụng đều kết nối về Node 3 cho mọi thao tác đọc/ghi dữ liệu — MongoDB làm kho lưu trữ chính và Redis phục vụ cache và đồng bộ sự kiện qua Streams giữa các microservice trên cả hai node.

Node 4 là node chuyên biệt cho việc thu thập và phân tích log, chạy toàn bộ ELK Stack trong Docker gồm Logstash, Elasticsearch và Kibana. Filebeat trên Node 1 và

Node 2 ship log về Logstash tại Node 4; Logstash parse và phân loại log trước khi lưu vào Elasticsearch, nếu log là WARNING hay ERROR sẽ gửi thông báo về chanel Discord thông qua Discord Webhook; Kibana cung cấp giao diện trực quan để tra cứu log, theo dõi lỗi và xây dựng dashboard vận hành theo thời gian thực. Việc tách logging ra node riêng đảm bảo hoạt động quan sát không tranh tài nguyên với các node ứng dụng, đặc biệt quan trọng khi hệ thống đang chịu tải cao.

CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE

3.1 Các microservice hoạt động

Hệ thống SunStack được thiết kế và phát triển dựa trên kiến trúc Microservices. Để tăng tính độc lập, khả năng mở rộng và khả năng bảo trì, toàn bộ hệ thống back-end được phân tách thành 9 dịch vụ thành phần. Mỗi dịch vụ phụ trách một miền nghiệp vụ độc lập và tuân thủ nguyên lý đơn trách nhiệm.

Giao tiếp nội bộ giữa các dịch vụ được thực hiện thông qua giao thức HTTP RESTful API đối với các tiến trình đồng bộ và thông qua Redis Streams đối với các tiến trình bất đồng bộ theo mô hình hướng sự kiện. Dưới đây là mô tả chức năng và vai trò của từng microservice trong hệ thống:

1. API Gateway Cổng giao tiếp API - Cổng 8000

Vai trò: API Gateway hoạt động như điểm truy cập duy nhất cho toàn bộ yêu cầu từ phía máy khách và giao diện front-end.

Chức năng: Thực hiện định tuyến yêu cầu đến các microservice nghiệp vụ tương ứng ở lớp phía sau. Ngoài chức năng định tuyến, dịch vụ còn đảm nhiệm điều phối giao tiếp giữa các thành phần và duy trì kết nối WebSocket nhằm hỗ trợ truyền tải dữ liệu thời gian thực.

2. Auth Service Dịch vụ xác thực và phân quyền - Cổng 8001

Vai trò: Đảm nhiệm chức năng bảo mật truy cập và quản lý phiên làm việc của hệ thống.

Chức năng: Xử lý các nghiệp vụ đăng ký, đăng nhập, phát hành và kiểm chứng JSON Web Token JWT. Dịch vụ hỗ trợ cơ chế xác thực thông qua bên thứ ba dựa trên tiêu chuẩn OAuth2, cụ thể đối với Google OAuth.

3. User Service Dịch vụ quản lý người dùng - Cổng 8002

Vai trò: Quản lý và lưu trữ dữ liệu định danh người dùng.

Chức năng: Cung cấp API phục vụ truy xuất và cập nhật thông tin hồ sơ cá nhân, đồng thời quản lý danh mục địa chỉ giao nhận hàng hóa.

4. Product Service Dịch vụ quản lý sản phẩm - Cổng 8003

Vai trò: Quản lý dữ liệu liên quan đến danh mục và sản phẩm của nền tảng thương mại điện tử.

Chức năng: Thực hiện các thao tác CRUD đối với sản phẩm và danh mục. Dịch vụ đồng thời tổng hợp và chuẩn hóa dữ liệu phục vụ hiển thị thông tin sản phẩm trên giao diện front-end.

5. Order Service Dịch vụ quản lý đơn hàng - Cổng 8004

Vai trò: Điều phối quy trình mua sắm và quản lý trạng thái vòng đời của đơn hàng.

Chức năng: Quản lý giỏ hàng, quy trình thanh toán và cập nhật trạng thái đơn hàng. Dịch vụ phát sinh các sự kiện trạng thái lên Message Broker nhằm đồng bộ dữ liệu tồn kho với Product Service.

6. Shop Service Dịch vụ quản lý cửa hàng - Cổng 8005

Vai trò: Quản trị các nghiệp vụ liên quan đến người bán.

Chức năng: Hỗ trợ tạo lập và cấu hình thông tin cửa hàng, đồng thời quản lý hoạt động bán hàng. Dịch vụ phát hành sự kiện khi dữ liệu cửa hàng thay đổi, tạo điều kiện đồng bộ bộ đệm cục bộ giữa các dịch vụ liên quan như Product Service và Order Service.

7. Payment Service Dịch vụ thanh toán - Cổng 8006

Vai trò: Xử lý độc lập các giao dịch thanh toán điện tử.

Chức năng: Khởi tạo URL thanh toán và tiếp nhận kết quả giao dịch từ đối tác thanh toán. Trong phạm vi dự án, dịch vụ được tích hợp với cổng thanh toán VNPay nhằm hỗ trợ xử lý giao dịch trực tuyến.

8. Chat Service Dịch vụ trò chuyện trực tuyến - Cổng 8010

Vai trò: Cung cấp hạ tầng giao tiếp trực tiếp giữa người mua và người bán.

Chức năng: Lưu trữ lịch sử tin nhắn, thông tin hội thoại và phối hợp với API Gateway trong quá trình phân phối tin nhắn theo thời gian thực đến đúng người dùng.

9. Upload Service Dịch vụ quản lý tệp tin đa phương tiện - Cổng 8011

Vai trò: Quản lý tài nguyên tĩnh và dữ liệu đa phương tiện của hệ thống.

Chức năng: Cung cấp API phục vụ tải lên và truy xuất tệp tin. Dữ liệu được cấu hình lưu trữ thông qua dịch vụ đám mây chuyên dụng như Google Cloud Storage, qua đó giảm tải yêu cầu lưu trữ và băng thông đối với máy chủ nội bộ.

Việc áp dụng kiến trúc microservices giúp tách biệt các miền dữ liệu và chức năng nghiệp vụ giữa các dịch vụ. Trong trường hợp một dịch vụ gặp sự cố, các dịch vụ còn lại vẫn có thể tiếp tục hoạt động độc lập, từ đó nâng cao tính sẵn sàng và khả năng chịu lỗi của toàn bộ hệ thống.

3.2 Xử lý khi service tạm thời không phản hồi

Trong kiến trúc phân tán nói chung và kiến trúc microservices nói riêng, việc một dịch vụ tạm thời không phản hồi do khởi động lại, quá tải hệ thống hoặc sự cố mạng nội bộ là tình huống thường gặp. Để tăng tính bền bỉ của hệ thống và hạn chế hiện tượng lỗi dây chuyền, hệ thống SunStack được thiết kế với nhiều cơ chế phòng vệ như sau:

1. Cơ chế kiểm soát thời gian chờ Timeout

Tất cả yêu cầu nội bộ giữa các dịch vụ thông qua giao thức HTTP đều được áp dụng chính sách giới hạn thời gian chờ. Tại API Gateway, HTTP Client được cấu hình với thời gian chờ tối đa xác định trước, ví dụ `timeout = 30.0s`. Cơ chế này bảo đảm tiến trình xử lý tại Gateway không bị treo vô thời hạn khi một downstream service mất phản hồi. Khi vượt quá thời gian chờ, kết nối được giải phóng và hệ thống trả phản hồi phù hợp cho phía máy khách, qua đó giảm nguy cơ cạn kiệt tài nguyên kết nối.

2. Cơ chế thử lại tự động Retry tại API Gateway

Khi API Gateway thực hiện chức năng proxy chuyển tiếp yêu cầu đến các dịch vụ nghiệp vụ, hệ thống áp dụng cơ chế Retry đối với các lỗi tạm thời. Trong trường hợp phát sinh các ngoại lệ như `ConnectError` hoặc `ReadTimeout`, Gateway tự động gửi lại yêu cầu đến dịch vụ đích sau một khoảng thời gian chờ tăng dần theo chiến lược backoff delay.

Nếu số lần thử vượt quá ngưỡng `max_attempts` và dịch vụ đích vẫn không phản hồi, Gateway sẽ dừng tiến trình xử lý và trả về mã trạng thái HTTP 503 Service

Unavailable. Cơ chế này hỗ trợ front-end nhận biết tình trạng quá tải hoặc gián đoạn tạm thời của hệ thống.

3. Cơ chế dự phòng Fallback kết hợp bộ nhớ đệm Cache

Tại các dịch vụ nghiệp vụ như Product Service hoặc Order Service, hệ thống ưu tiên sử dụng dữ liệu từ bộ nhớ đệm nội bộ khi cần truy xuất dữ liệu chéo giữa các dịch vụ. Thay vì phụ thuộc hoàn toàn vào giao tiếp HTTP trực tiếp, dữ liệu được đồng bộ liên tục thông qua Redis Streams và lưu trữ tại Redis Cache.

Trường hợp Cache Hit: Dữ liệu được truy xuất trực tiếp từ bộ nhớ đệm với độ trễ thấp, đồng thời loại bỏ rủi ro phát sinh do giao tiếp mạng nội bộ.

Trường hợp Cache Miss hoặc downstream service gặp sự cố: Hệ thống thực hiện truy vấn HTTP dự phòng đến dịch vụ đích. Nếu truy vấn tiếp tục thất bại, ngoại lệ sẽ được xử lý và hệ thống chủ động trả về giá trị mặc định hoặc giá trị rỗng. Cơ chế này giúp duy trì khả năng phản hồi của hệ thống ngay cả khi một số dịch vụ phụ trợ tạm thời không khả dụng.

Sự kết hợp giữa cơ chế Timeout, Retry và Fallback giúp hệ thống microservices duy trì tính ổn định và khả năng chịu lỗi trong điều kiện vận hành thực tế. Các cơ chế này góp phần hạn chế ảnh hưởng lan truyền giữa các dịch vụ và duy trì khả năng phục vụ ngay cả khi xuất hiện sự cố cục bộ trong hệ thống.

CHƯƠNG 4. THIẾT KẾ RESTFUL API

4.1 Nguyên tắc thiết kế api chung

- Hệ thống tuân thủ nghiêm ngặt chuẩn kiến trúc RESTful.
- Toàn bộ API được định tuyến tập trung qua API Gateway, sử dụng phiên bản thống nhất (tiền tố /api/v1/).
- Định dạng dữ liệu trao đổi mặc định: JSON. Các mã trạng thái HTTP (200, 201, 400, 401, 403, 404, 500) được chuẩn hóa cho toàn bộ các dịch vụ.

4.2 Kiến trúc phân hệ API

Hệ thống được tổ chức theo kiến trúc microservices với sự phân tách rõ ràng giữa các miền nghiệp vụ. Để bảo đảm tính thống nhất trong quá trình giao tiếp, toàn bộ yêu cầu từ phía máy khách đều được định tuyến tập trung thông qua API Gateway. Cách tiếp cận này giúp tách biệt cấu trúc mạng nội bộ khỏi phía client, đồng thời chuẩn hóa cơ chế xử lý lỗi và hỗ trợ quản lý tài nguyên theo từng miền nghiệp vụ độc lập.

Bảng tổng quan dưới đây mô tả danh mục các dịch vụ cùng những nhóm nghiệp vụ cốt lõi do phân hệ API của hệ thống đảm nhiệm

Bảng 4.1 Danh mục các vi dịch vụ và nhóm nghiệp vụ cốt lõi

STT	Dịch vụ	Cụm nghiệp vụ đảm nhận	Đối tượng tương tác / Vai trò chính
1	Gateway Service	- Điểm vào tập trung của toàn bộ hệ thống back-end. - Thực hiện định tuyến request đến các vi dịch vụ phía sau. - Quản lý kết nối và hỗ trợ các chức năng thời gian thực.	Máy khách Front-end, hệ thống vi dịch vụ nội bộ

2	Auth Service	- Quản lý đăng ký, đăng nhập, đăng xuất và thay đổi mật khẩu. - Cấp phát, kiểm tra và làm mới JWT. - Tích hợp xác thực bên thứ ba thông qua Google OAuth2.	Người dùng hệ thống, hệ thống vi dịch vụ nội bộ
3	User Service	- Quản lý thông tin hồ sơ tài khoản người dùng. - Quản lý danh mục địa chỉ nhận hàng và thiết lập mặc định.	Người mua, người bán
4	Shop Service	- Tạo mới và quản lý hồ sơ cửa hàng cùng trạng thái hoạt động. - Cung cấp công cụ quản trị dành cho người bán như dashboard tổng quan và danh sách sản phẩm. - Phát sự kiện đồng bộ dữ liệu cửa hàng sang hệ thống cache.	Chủ cửa hàng Seller, Product Service, Order Service
5	Product Service	- Quản lý thông tin sản phẩm, danh mục sản phẩm và biến thể SKU. - Cung cấp dữ liệu phục vụ hiển thị trang chi tiết sản phẩm. - Đồng bộ thông tin cửa hàng từ cache nhằm tối ưu hiệu năng truy xuất.	Người mua, người bán, hệ thống vi dịch vụ nội bộ
6	Order Service	- Quản lý giỏ hàng của người dùng.	Người mua, người bán, Payment Service

		- Xử lý luồng đặt hàng, khởi tạo đơn hàng tổng và đơn hàng theo từng cửa hàng. - Quản lý lịch sử và cập nhật trạng thái vòng đời đơn hàng.	
7	Payment Service	- Khởi tạo bản ghi giao dịch và quản lý trạng thái thanh toán tương ứng với đơn hàng. - Tích hợp cổng thanh toán VNPay để lấy URL thanh toán và xử lý callback phản hồi.	Người mua, Order Service, cổng thanh toán VNPay
8	Chat Service	- Quản lý danh sách phòng chat, trạng thái tin nhắn chưa đọc và lịch sử hội thoại. - Xử lý luồng gửi nhận tin nhắn thời gian thực giữa các tài khoản.	Người mua, người bán
9	Upload Service	- Tiếp nhận và xử lý tải lên các tệp đa phương tiện bao gồm hình ảnh và video. - Quản lý lưu trữ tệp thông qua dịch vụ lưu trữ đám mây.	Toàn bộ dịch vụ cần lưu trữ tài nguyên đa phương tiện

Để phục vụ hoạt động kiểm thử, phát triển và phân tích kỹ thuật, toàn bộ thông tin đặc tả về phương thức truyền tin HTTP, đường dẫn định tuyến và cấu trúc dữ liệu của từng endpoint đã được tách riêng và hệ thống hóa tại Phụ lục C: Đặc tả chi tiết Endpoint API ở cuối tài liệu.

CHƯƠNG 5. LUỒNG MESSAGE QUEUE (REDIS STREAMS)

Trong kiến trúc microservices của nền tảng thương mại điện tử SunStack, cơ chế giao tiếp đồng bộ thông qua HTTP REST API có thể làm gia tăng độ trễ xử lý, tạo điểm nghẽn hiệu năng và ảnh hưởng đến tính sẵn sàng của hệ thống. Để giảm sự phụ thuộc trực tiếp giữa các dịch vụ, hệ thống triển khai Redis Streams như một hạ tầng truyền thông điệp phục vụ giao tiếp bất đồng bộ giữa các thành phần nghiệp vụ.

5.1 Cơ sở lý thuyết về Redis Streams và Consumer Group

Redis Streams là một cấu trúc dữ liệu dạng nhật ký sự kiện cho phép lưu trữ các thông điệp theo trình tự thời gian. Mỗi thông điệp được gán một mã định danh duy nhất, thường dựa trên timestamp tại thời điểm phát sinh sự kiện.

Trong kiến trúc microservices, Redis Streams hỗ trợ cơ chế Consumer Group nhằm phân phối thông điệp giữa nhiều consumer thuộc cùng một nhóm xử lý.

- Thay vì tất cả consumer cùng nhận một thông điệp theo mô hình broadcast, Consumer Group cho phép phân bổ tải xử lý giữa các instance khác nhau của cùng một dịch vụ.
- Khi một consumer đọc thông điệp, bản ghi sẽ được chuyển sang trạng thái pending. Chỉ khi consumer xử lý thành công và gửi lệnh xác nhận XACK, thông điệp mới được đánh dấu hoàn tất.
- Cơ chế này giúp bảo đảm tính toàn vẹn dữ liệu và hạn chế nguy cơ mất sự kiện khi hệ thống phát sinh lỗi trong quá trình xử lý.

5.2 Kiến trúc Cross-service Cache bằng Streams

Một trong những thách thức chính của kiến trúc microservices là truy xuất dữ liệu chéo giữa các dịch vụ. Ví dụ, Order Service cần truy xuất thông tin cửa hàng từ Shop Service trong quá trình khởi tạo đơn hàng. Nếu toàn bộ truy vấn đều thực hiện thông qua HTTP trực tiếp, hệ thống dễ phát sinh độ trễ và phụ thuộc chặt giữa các dịch vụ.

Để giải quyết vấn đề này, hệ thống triển khai cơ chế Cross-service Cache tại module backend/libs/redis/service_cache.py.

5.2.1 Cấu trúc bộ nhớ đệm đa tầng

Hệ thống sử dụng mô hình bộ nhớ đệm hai tầng nhằm tối ưu thời gian phản hồi và giảm tải truy vấn mạng nội bộ:

1. Tầng Local Memory Cache

Dữ liệu được lưu trực tiếp trong bộ nhớ RAM của tiến trình ứng dụng thông qua cấu trúc dictionary. Tầng này cung cấp tốc độ truy xuất gần như tức thời với độ phức tạp truy cập $O(1)$.

2. Tầng Distributed Cache Redis

Dữ liệu được tuần tự hóa dưới dạng JSON và lưu trên Redis với thời gian sống mặc định $TTL = 3600$ giây. Tầng cache này đóng vai trò kho lưu trữ dùng chung giữa các dịch vụ và duy trì dữ liệu khi tiến trình cục bộ được khởi động lại.

5.2.2 Luồng hoạt động theo mô hình hướng sự kiện

- Source of Truth: Khi dữ liệu thay đổi, dịch vụ sở hữu dữ liệu phát một sự kiện vào stream dùng chung thông qua lệnh XADD.
- Consumer: Các dịch vụ khác đăng ký lắng nghe stream thông qua Consumer Group có tên service-cache. Sau khi nhận sự kiện bằng lệnh XREADGROUP, hệ thống phân tích payload và cập nhật lại dữ liệu trong Redis Cache và Local Cache.

5.3 Chi tiết các phân luồng và cấu trúc payload

Để bảo đảm tính module hóa và tách biệt nghiệp vụ, hệ thống phân chia các stream theo từng nhóm chức năng chuyên biệt.

5.3.1 Stream đồng bộ cache cửa hàng

- Tên Stream: service:shop:updates
- Mục đích: Đồng bộ dữ liệu cửa hàng đến các dịch vụ như Product Service và Order Service.

Ví dụ cấu trúc payload:

{

```
"action": "create|update|delete",  
  
"shop_id": "shop_12345",  
  
"data": {  
  
  "id": "shop_12345",  
  
  "name": "SunStack Official Store",  
  
  "user_id": "user_789"  
  
}  
  
}
```

Khi action mang giá trị update, hệ thống cập nhật key theo định dạng cache:shop:<shop_id> trên Redis Cache và Local Cache. Ngoài ra, hệ thống duy trì thêm secondary index cache:shop:by_user:<user_id> nhằm tối ưu truy vấn cửa hàng theo mã người dùng.

5.3.2 Stream đồng bộ cache sản phẩm

- Tên Stream: service:product:updates
- Mục đích: Đồng bộ dữ liệu sản phẩm giữa các dịch vụ trong hệ thống.

Luồng xử lý tương tự stream cửa hàng. Tuy nhiên, khi action mang giá trị delete, hệ thống thực hiện cơ chế invalidate nhằm xóa toàn bộ key liên quan khỏi cả hai tầng cache để ngăn việc sử dụng dữ liệu lỗi thời.

5.3.3 Stream sự kiện đơn hàng

Stream này không phục vụ lưu cache mà được sử dụng cho điều phối quy trình nghiệp vụ theo mô hình Saga quy mô nhỏ.

- Tên Stream: order-events
- Các sự kiện tiêu biểu:
 - order.created
 - order.updated
 - order.cancelled

Khi khách hàng tạo đơn hàng thành công, Order Service phát sự kiện `order.created`. Sau đó, Product Service tiếp nhận sự kiện và cập nhật lại số lượng tồn kho tương ứng. Cách tiếp cận này giúp giảm thời gian chờ phản hồi đối với phía người dùng cuối.

5.4 Cơ chế chịu lỗi và Dead Letter Queue

Sự cố mạng hoặc lỗi dữ liệu có thể làm gián đoạn quá trình xử lý sự kiện của consumer. Để tăng khả năng chịu lỗi, hệ thống triển khai hai cơ chế bảo vệ:

1. Cơ chế Retry

Consumer chỉ gửi xác nhận XACK khi quá trình xử lý hoàn tất thành công. Nếu phát sinh lỗi, hệ thống thực hiện retry theo chiến lược backoff trong số vòng lặp giới hạn.

2. Dead Letter Queue DLQ

Nếu số lần retry vượt quá ngưỡng cho phép, thông điệp lỗi sẽ được chuyển sang stream phụ có hậu tố `:dlq`, ví dụ `service:shop:updates:dlq`.

Cơ chế này giúp stream chính tiếp tục hoạt động mà không bị chặn bởi các thông điệp lỗi. Đồng thời, đội ngũ vận hành có thể truy xuất DLQ để phân tích nguyên nhân và xử lý lại sau khi khắc phục sự cố.

5.5 Đánh giá lựa chọn Redis Streams so với Pub/Sub

Redis Pub/Sub thường được sử dụng trong các hệ thống nhỏ với yêu cầu truyền sự kiện đơn giản. Tuy nhiên, hệ thống SunStack lựa chọn Redis Streams do các ưu điểm phù hợp hơn với kiến trúc microservices.

- Tính lưu vết thông điệp: Pub/Sub hoạt động theo mô hình phát và quên. Nếu consumer không hoạt động tại thời điểm phát sự kiện, dữ liệu sẽ bị mất. Redis Streams cho phép lưu trữ thông điệp và hỗ trợ consumer đọc lại các sự kiện chưa xử lý.
- Khả năng mở rộng theo chiều ngang: Consumer Group hỗ trợ phân phối thông điệp giữa nhiều instance của cùng một dịch vụ, qua đó tăng khả năng scale-out khi tải hệ thống tăng cao.

- Tối ưu hạ tầng triển khai: Redis đã được sử dụng đồng thời cho cache và message queue, giúp giảm độ phức tạp vận hành so với việc triển khai thêm các hệ thống chuyên biệt như Kafka hoặc RabbitMQ.

Nhờ các đặc điểm trên, Redis Streams phù hợp với yêu cầu đồng bộ dữ liệu, điều phối sự kiện và bảo đảm khả năng chịu lỗi trong kiến trúc microservices của hệ thống SunStack.

CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1 Môi trường triển khai và thử nghiệm

Để đảm bảo tính nhất quán, độc lập và dễ dàng tái tạo môi trường, toàn bộ hệ thống SunStack được đóng gói bằng công cụ Docker. Quá trình triển khai cục bộ được quản lý tập trung thông qua Docker Compose, cho phép khởi chạy toàn bộ cụm dịch vụ chỉ với một tệp cấu hình duy nhất.

Cấu hình môi trường thử nghiệm chi tiết:

- Hệ điều hành nền tảng: Windows có tích hợp Docker Engine và Docker Compose.
- Kiến trúc mạng nội bộ (Docker Network): Các container được cô lập và giao tiếp với nhau qua mạng ảo mang tên `sunstack_network`. Nhằm đảm bảo bảo mật, API Gateway (chạy tại cổng 8000) là điểm giao tiếp duy nhất mở cổng giao tiếp ra bên ngoài để tiếp nhận yêu cầu từ ứng dụng Frontend (chạy ở cổng 5173). Không có Microservice nào được truy cập trực tiếp từ Internet.
- Hệ sinh thái dịch vụ hạ tầng:
 - MongoDB: Cơ sở dữ liệu phi cấu trúc chính, mỗi service sở hữu một database logic riêng biệt.
 - Redis: Đóng vai trò bộ nhớ đệm để tăng tốc độ truy xuất và sử dụng cấu trúc Redis Streams làm Message Broker cho kiến trúc Event-driven.
 - Elastic Filebeat: Tác nhân hạng nhẹ dùng để thu gom và vận chuyển log từ các container về hệ thống lưu trữ tập trung.

Quá trình khởi động toàn bộ cụm hệ thống (bao gồm Frontend, Gateway, 8 Microservices lõi và các dịch vụ hạ tầng) được tự động hóa hoàn toàn thông qua tập lệnh `docker compose up -d`. Bên cạnh đó, stack ELK được triển khai được thu thập log từ các service cũng được khởi tạo thông qua lệnh `docker compose -f docker-compose-elk.yml`.

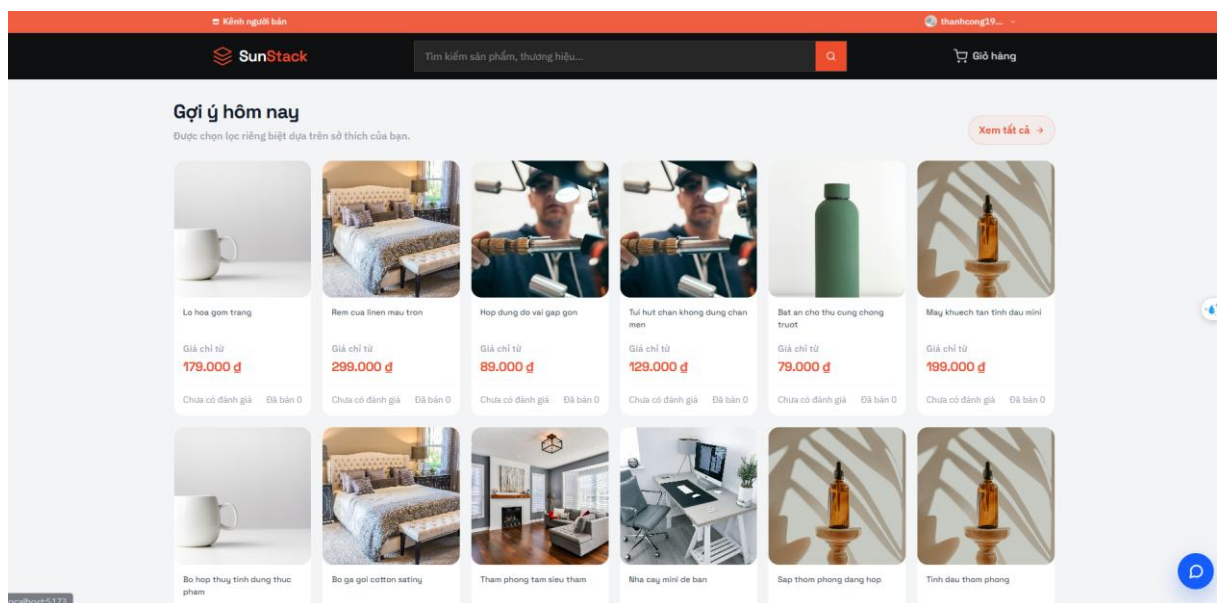
Hệ thống cũng được deploy lên cloud trên Google Cloud Platform với 4 VM tượng trưng cho 4 node:

- Node 1, node 2: Khởi chạy 2 thực thể ứng dụng SunStack nhằm đảm bảo tính sẵn sàng cao và khả năng chia tải.
- Node 3: Triển khai hệ quản trị cơ sở dữ liệu MongoDB, cấu hình làm Shared Database cho cả hai ứng dụng.
- Node 4: Cấu hình hệ thống ELK Stack (Elasticsearch, Logstash, Kibana) chịu trách nhiệm thu thập, lưu trữ tập trung và trực quan hóa dữ liệu log. Đồng thời, node này được tích hợp cơ chế giám sát để tự động cảnh báo lỗi hệ thống về kênh Discord qua Webhook.

Bên cạnh đó, để tối ưu hóa trải nghiệm người dùng và định tuyến tập trung về một tên miền duy nhất (sunstack.org), nhóm đã tích hợp dịch vụ Google Cloud Load Balancing. Bộ cân bằng tải này đóng vai trò tiếp nhận traffic từ Internet, sau đó điều phối thông minh đến địa chỉ IP của cụm Application (Node 1 và Node 2).

6.2 Kịch bản kiểm thử

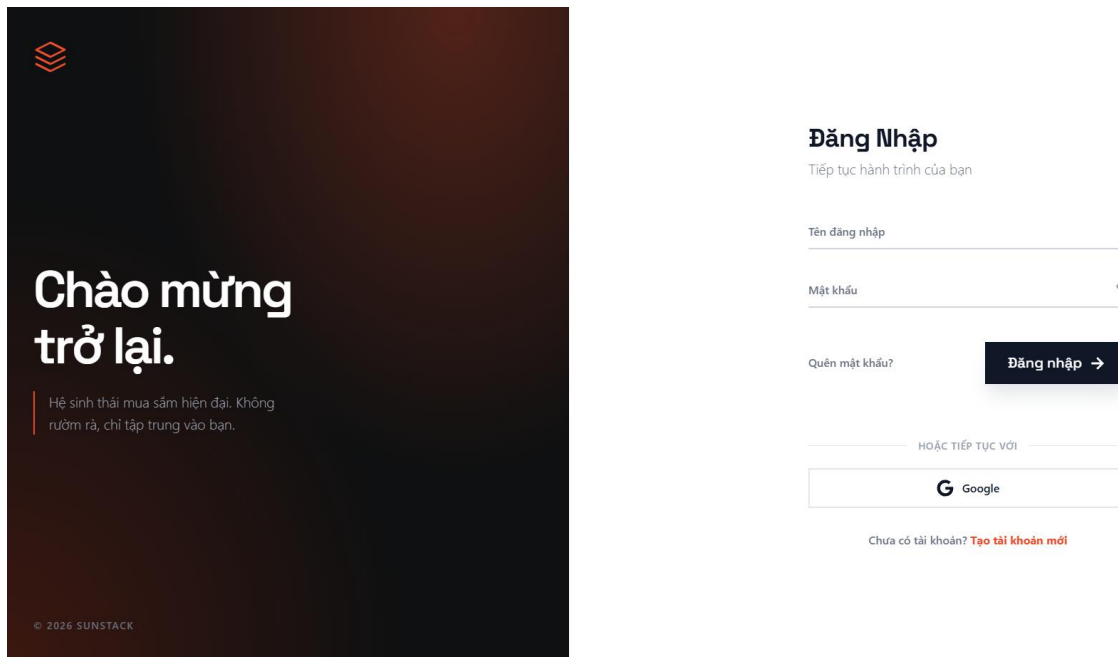
Quá trình kiểm thử được chia thành hai hạng mục chính: kiểm thử tính đúng đắn của luồng nghiệp vụ và kiểm thử tính bền bỉ của kiến trúc phân tán.



Hình 6.1: Giao diện trang chủ

6.2.1 Kiểm thử chức năng nghiệp vụ

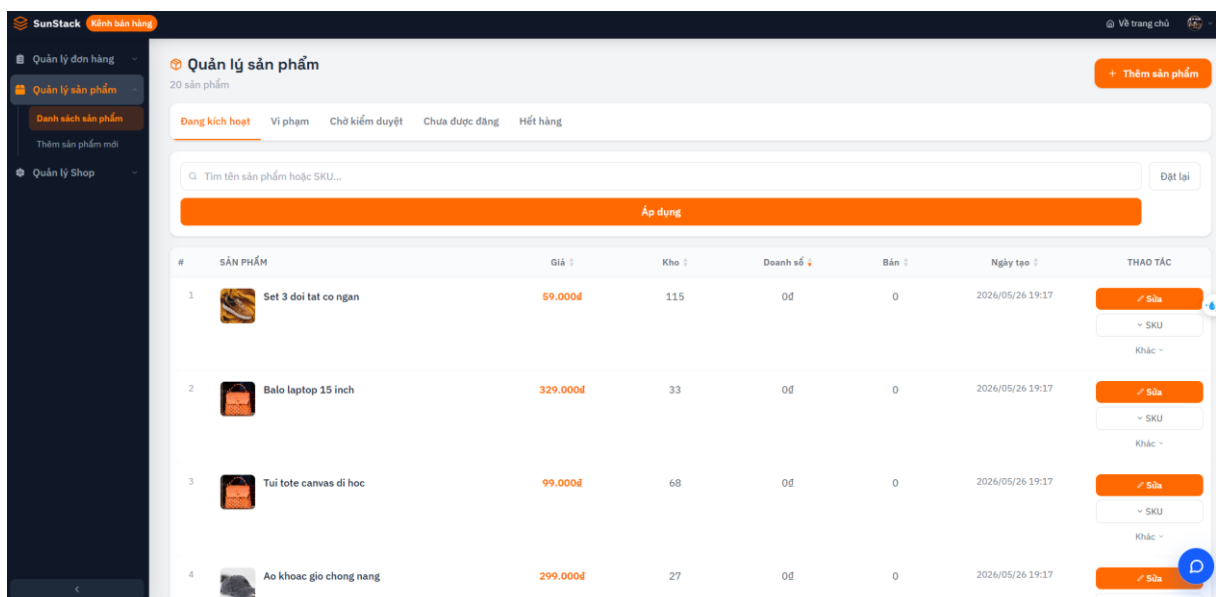
Luồng xác thực và quản lý định danh:



Hình 6.2: Giao diện đăng nhập

- Kịch bản: Đăng ký, xác thực email giả lập và đăng nhập để nhận chuỗi JWT.
- Đánh giá phân quyền: Cố tình sử dụng tài khoản khách hàng để gọi API tạo sản phẩm dành riêng cho Seller.
- Kết quả mong đợi: Hệ thống chặn yêu cầu ngay tại auth-service, trả về mã lỗi HTTP 403 (Forbidden).

Luồng quản lý gian hàng và vòng đời sản phẩm:



Hình 6.3: Giao diện quản lý sản phẩm của shop

- Kịch bản: Người bán khởi tạo hồ sơ Shop, tải ảnh lên thông qua upload-service, sau đó thiết lập sản phẩm với nhiều biến thể SKU với cấu hình giá và số lượng khác nhau.
- Kết quả mong đợi: Dữ liệu sản phẩm và SKU được lưu trữ toàn vẹn, liên kết chính xác với định danh của Shop.

Luồng giao dịch và thanh toán điện tử:

The screenshot displays the checkout interface of the SunStack application. It is divided into several sections:

- Thanh toán (Payment):** Contains a product card for "Bat an cho thu cung chong truot" (Anti-slip mat) with a price of 99,000 VND. Below it, a summary table shows: Tổng sản phẩm (Total products) 99,000 VND, Phí vận chuyển (Shipping fee) 25,000 VND, and Ngày giao dự kiến (Estimated delivery date) 27/5/2026.
- Địa chỉ (Address):** Displays the user's address: Nguyễn Thành Công (0974432848), adsfg, Xã Hòa Thuận, Thành phố Buôn Ma Thuột, Tỉnh Đắk Lắk. A "Thay đổi" (Change) link is available.
- Hình thức thanh toán (Payment Method):** Offers two options: "Thanh toán tiền mặt" (Cash payment) and "Chuyển khoản" (Bank transfer), with the latter being selected.
- Đơn hàng (Order):** A summary of the order totals: Tổng tiền sản phẩm (Total product price) 99,000 VND, Phí vận chuyển (Shipping fee) 25,000 VND, and a bolded Tổng cộng (Total) of 124,000 VND. A "Đặt hàng" (Place order) button is at the bottom.

Hình 6.4: Giao diện thanh toán

- Kịch bản: Khách hàng thêm sản phẩm vào giỏ và tiến hành thanh toán.
- Thực thi: Hệ thống điều phối lệnh gọi API nội bộ từ order-service sang shop-service và product-service để kiểm tra tồn kho và tính toán tổng giá trị đơn hàng. Sau đó, hệ thống chuyển hướng người dùng sang môi trường thử nghiệm của VNPay.



Giao dịch hết hạn sau

14

:

29

Thông tin đơn hàng (Test)

Số tiền thanh toán

124.000VND

Giá trị đơn hàng

124.000VND

Phí giao dịch

0VND

Mã đơn hàng

6a16074ff0d3d82c9dc30330

Nhà cung cấp

<https://vnshop.vn/>

Thanh toán qua Ngân hàng NCB

Thẻ nội địa

Số thẻ



Tên chủ thẻ

Ngày phát hành ⓘ

Mã khuyến mại


[Điều kiện sử dụng dịch vụ](#)

Hủy thanh toán

Tiếp tục


hotrovnpay@vnpay.vn

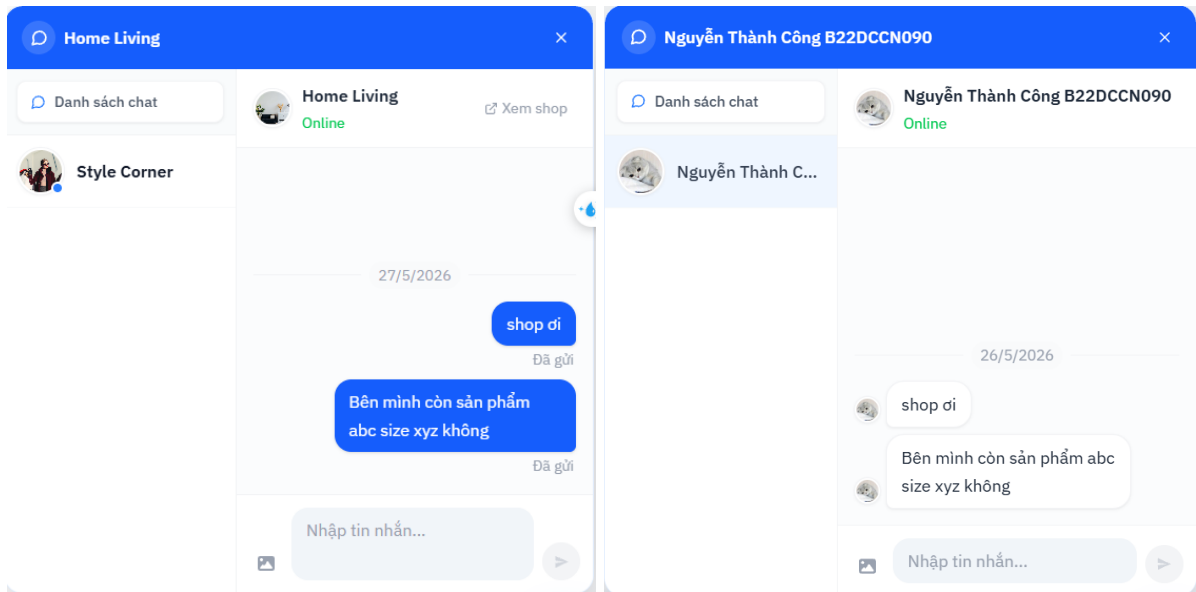



Hình 6.5: Giao diện thanh toán điện tử với VNPAY

- Kết quả: Sau khi thanh toán giả lập thành công, VNPAY gọi Webhook về payment-service. Dịch vụ này phát ra một sự kiện để order-service cập nhật trạng thái đơn hàng thành "Đã thanh toán" một cách tự động.

Luồng tương tác thời gian thực:

27

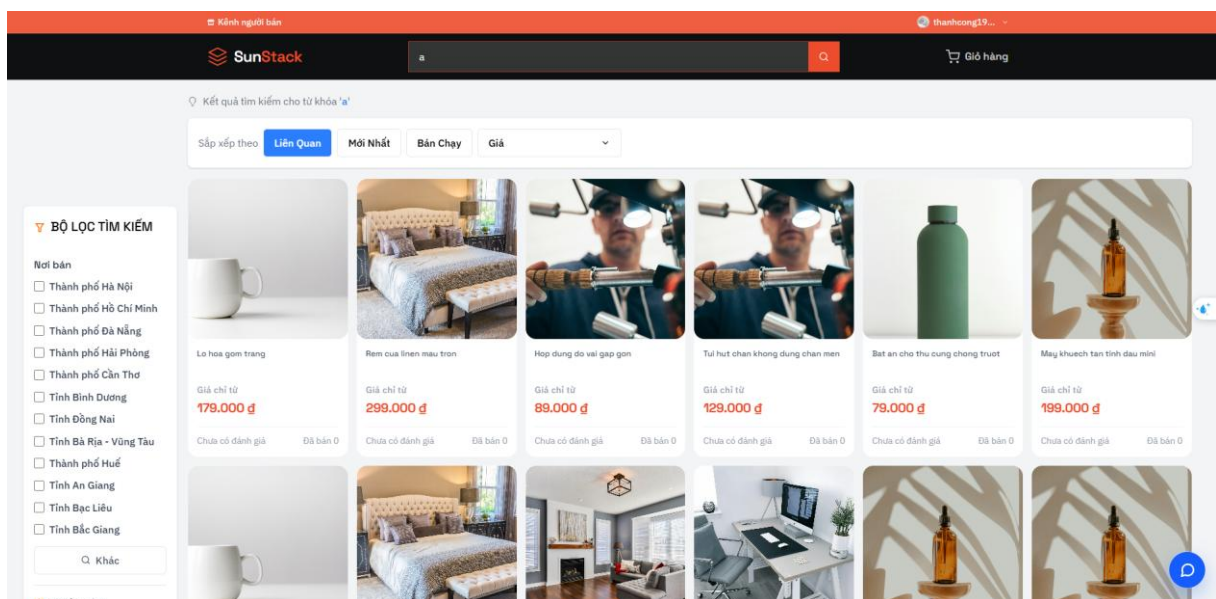


Hình 6.6: Giao diện chat realtime.

- Kịch bản: Giao tiếp trực tiếp giữa hai phiên trình duyệt, một bên là khách hàng và bên còn lại là chủ Shop.
- Thực thi: Dữ liệu tin nhắn được truyền qua giao thức WebSocket thông qua API Gateway.
- Kết quả: Tin nhắn xuất hiện ngay lập tức trên màn hình người nhận mà không cần tải lại trang.

6.2.2 Kiểm thử chức năng chịu lỗi và tính bền bỉ

Kịch bản gián đoạn cục bộ (Partial Outage):



Hình 6.7: Giao diện tìm kiếm sản phẩm

- Thực thi: Tắt đột ngột product-service thông qua việc sử dụng lệnh docker stop và truy cập trang chủ E-commerce.
- Kết quả: Nhờ cơ chế Caching, trang chủ vẫn hiển thị danh sách sản phẩm được lấy từ Redis thay vì báo lỗi toàn hệ thống. Điều này chứng minh hệ thống không mắc lỗi Single Point of Failure.

Kịch bản tự động thử lại:

- Thực thi: Giả lập độ trễ cao tại order-service. API Gateway được cấu hình cơ chế tự động thử lại tối đa 2 lần khi gặp lỗi Timeout.
- Kết quả: Ở lần gọi thứ 2, yêu cầu được xử lý thành công. Người dùng cuối chỉ cảm thấy độ trễ tăng lên đôi chút thay vì nhận màn hình thông báo lỗi hệ thống.

Kịch bản xử lý thông điệp độc hại với DLQ:

- Giải thích thuật ngữ: DLQ là một hàng đợi đặc biệt dùng để chứa các message không thể xử lý thành công sau nhiều lần thử lại, giúp ngăn chặn tình trạng một message lỗi gây kẹt toàn bộ hệ thống xử lý.
- Thực thi: Cố tình thêm một message sai định dạng vào Redis Streams. Consumer đọc được nhưng văng lỗi.
- Kết quả: Sau số lần thử lại tối đa, hệ thống tự động dời message này sang hàng đợi, dlq sẽ được đánh giá thủ công sau, luồng xử lý chính vẫn tiếp tục hoạt động trơn tru.

6.3 Đánh giá ưu điểm của hệ thống

Quá trình triển khai thực tiễn đã khẳng định tính đúng đắn khi lựa chọn kiến trúc Microservices cho SunStack:

- Tách biệt mối quan tâm: Dựa trên phương pháp Thiết kế hướng tên miền, mỗi dịch vụ chỉ giải quyết một bài toán nghiệp vụ duy nhất. Mã nguồn trở nên gọn nhẹ, dễ đọc. Sự cố tại chat-service được cô lập hoàn toàn, không gây ảnh hưởng đến luồng giao dịch cốt lõi của order-service.
- Khả năng mở rộng linh hoạt: Trong các kịch bản thực tế như sự kiện "Siêu Sale", lưu lượng tạo đơn hàng có thể tăng vọt. Quản trị viên chỉ cần nhân bản riêng lẻ

container của order-service để đáp ứng tải, tiết kiệm tối đa tài nguyên so với việc nhân bản toàn bộ một ứng dụng nguyên khối.

- Tối ưu hóa hiệu năng với luồng sự kiện: Các tác vụ nặng hoặc không yêu cầu tính đồng bộ tức thời (như gửi email thông báo, dọn dẹp bộ nhớ đệm, tính toán log) được đẩy vào Redis Streams để xử lý ngầm. Nhờ đó, thời gian phản hồi API trả về cho người dùng được rút ngắn đáng kể.

6.4 Hạn chế và hướng phát triển trong tương lai

Trong khuôn khổ đề tài, hệ thống đã hoàn thiện các tính năng cốt lõi. Tuy nhiên, để đạt tiêu chuẩn của một nền tảng vận hành trong môi trường thực tế, hệ thống cần được nghiên cứu và mở rộng ở các khía cạnh sau:

- Quản lý giao dịch phân tán:
 - Hạn chế: Hiện tại, thao tác hoàn trả số lượng tồn kho khi đơn hàng bị hủy vẫn sử dụng cơ chế gọi API đồng bộ đơn giản hoặc bắn event cơ bản.
 - Giải pháp tương lai: Áp dụng triệt để mô hình Saga Pattern tức là quản lý chuỗi các giao dịch cục bộ và tự động gọi lệnh bù đắp - Rollback khi có lỗi kết hợp với Outbox Pattern: lưu sự kiện vào chung cơ sở dữ liệu với nghiệp vụ chính trước khi gửi đi để đảm bảo tính toàn vẹn dữ liệu tuyệt đối chuẩn ACID trong môi trường phân tán.
- Tối ưu hóa công cụ tìm kiếm:
 - Hạn chế: Hoạt động tìm kiếm sản phẩm đang dựa vào các truy vấn trực tiếp trên MongoDB, dễ gây quá tải khi lượng sản phẩm lên đến hàng triệu.
 - Giải pháp tương lai: Triển khai công cụ tìm kiếm chuyên dụng Elasticsearch. Dữ liệu sẽ được đồng bộ theo thời gian thực từ MongoDB sang Elasticsearch thông qua kỹ thuật CDC sử dụng Debezium hoặc Monstache.
- Hoàn thiện hệ thống giám sát và truy vết:
 - Hạn chế: Hệ thống mới chỉ dừng ở mức thu thập log phân tán thông qua Filebeat.

- Giải pháp tương lai: Tích hợp các công cụ truy vết phân tán như Jaeger hoặc Zipkin để theo dõi hành trình của một request đi qua hàng loạt microservices. Đồng thời, thiết lập hệ thống cảnh báo và biểu đồ giám sát tài nguyên bằng bộ đôi Prometheus & Grafana.
- Tự động hóa luồng triển khai: Thiết lập quy trình tích hợp và triển khai liên tục thông qua việc sử dụng GitHub Actions hoặc GitLab CI để tự động chạy Unit Test, đóng gói Docker Image và deploy hệ thống lên cụm Kubernetes trên Cloud, đảm bảo khả năng vận hành chuyên nghiệp và tự động hóa cao.

TÀI LIỆU THAM KHẢO

- [1] Anusha Reddy Guntakandla, “Microservices Architecture: Decomposing E-Commerce Monoliths into Scalable, Independent Services,” *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 58s, pp. 642–650, Aug. 2025, doi: 10.52783/jisem.v10i58s.12665.
- [2] Swamy Sai Krishna Kireeti Athamakuri and Jagadeesh Thiruveedula, “Microservices Architecture in E-commerce: A Comparative Analysis of Performance, Scalability, and Maintainability,” *Int. J. Res. Publ. Semin.*, vol. 16, no. 2, pp. 110–124, Apr. 2025, doi: 10.36676/jrps.v16.i2.56.
- [3] Sandeep Kumar, “Event-Driven Microservices Architectures: Principles, Patterns and Best Practices,” *World J. Adv. Eng. Technol. Sci.*, vol. 15, no. 3, pp. 2109–2117, Jun. 2025, doi: 10.30574/wjaets.2025.15.3.1137.
- [4] H. Molchanov and A. Zhmaiev, “CIRCUIT BREAKER IN SYSTEMS BASED ON MICROSERVICES ARCHITECTURE,” *Adv. Inf. Syst.*, vol. 2, no. 4, pp. 74–77, Dec. 2018, doi: 10.20998/2522-9052.2018.4.13.
- [5] C. Pahl, “Containerization and the PaaS Cloud,” *IEEE Cloud Comput.*, vol. 2, no. 3, pp. 24–31, May 2015, doi: 10.1109/MCC.2015.51.
- [6] R. Ali, “Scalable and Secure AI Inference in Healthcare: A Comparative Benchmarking of FastAPI and Triton Inference Server on Kubernetes,” 2026, *arXiv*. doi: 10.48550/ARXIV.2602.00053.
- [7] R. Maharjan, M. S. H. Chy, M. A. Arju, and T. Cerny, “Benchmarking Message Queues,” *Telecom*, vol. 4, no. 2, pp. 298–312, Jun. 2023, doi: 10.3390/telecom4020018.
- [8] M. Rathore and S. S. Bagui, “MongoDB: Meeting the Dynamic Needs of Modern Applications,” *Encyclopedia*, vol. 4, no. 4, pp. 1433–1453, Sep. 2024, doi: 10.3390/encyclopedia4040093.
- [9] Ms. Monika Tripathi, Ms. Dimpal Jain, Ms. Khushboo Sharma, Ms. Anuradha, and Mr. Daulatram, “Web Development Framework,” *Int. J. Adv. Res. Sci. Commun. Technol.*, pp.

PHỤ LỤC

Mã nguồn dự án: <https://github.com/sybui2004/CHTPT>



PHỤ LỤC A: ĐẶC TẢ CHI TIẾT CÁC ENDPOINT API.

Phụ lục này hệ thống hóa toàn bộ các đường dẫn định tuyến (endpoints), phương thức truyền tin tương ứng (HTTP Methods) và vai trò xử lý nghiệp vụ của từng vi dịch vụ thuộc hệ thống dựa trên thông tin ghi nhận thực tế từ mã nguồn.

A.1. Gateway Service

Dịch vụ đảm nhận vai trò định tuyến tập trung và xử lý các tác vụ kiểm tra trạng thái hoặc điều phối thông điệp thời gian thực.

Bảng A.1: Đặc tả API của Gateway Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/health	Kiểm tra trạng thái hoạt động của API Gateway.
POST	/internal/push_message	Endpoint nội bộ cho phép các vi dịch vụ gửi thông điệp đến WebSocket client thông qua Gateway.

A.2. Auth Service

Tập hợp các API phục vụ quản lý trạng thái xác thực và tài khoản định danh người dùng hệ thống dưới tiền tố /api/v1/auth.

Bảng A.2: Đặc tả API của Auth Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
POST	/api/v1/auth/login	Thực hiện đăng nhập tài khoản vào hệ thống.
POST	/api/v1/auth/register	Thực hiện đăng ký tài khoản người dùng mới.

POST	/api/v1/auth/logout	Đăng xuất và hủy phiên làm việc hiện tại.
POST	/api/v1/auth/refresh	Thực hiện làm mới token bằng mô hình dữ liệu TokenResponse.
GET	/api/v1/auth/refreshToken	Truy xuất thông tin mã làm mới phiên thông qua mô hình dữ liệu RefreshTokenResponse.
POST	/api/v1/auth/change-password	Thay đổi mật khẩu của tài khoản người dùng.

A.3. User Service

Các endpoint đảm nhận nhiệm vụ xử lý thông tin cá nhân và quản lý cấu trúc số địa chỉ giao nhận của người dùng hệ thống.

Bảng A.3: Đặc tả API của User Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/api/v1/users/me	Lấy dữ liệu chi tiết hồ sơ tài khoản cá nhân hiện tại.
PUT	/api/v1/users/me	Cập nhật thông tin hồ sơ tài khoản cá nhân hiện tại.
GET	/api/v1/users/addresses	Truy xuất toàn bộ danh sách địa chỉ nhận hàng của người dùng.
POST	/api/v1/users/addresses	Khởi tạo và lưu trữ địa chỉ nhận hàng mới.
PUT	/api/v1/users/addresses/{address_id}	Cập nhật thông tin chi tiết của địa chỉ nhận hàng theo mã định danh address_id.

A.4. Shop Service

Hệ thống API được phân tách thành hai nhóm chính gồm nhóm REST phục vụ tra cứu thông tin công khai và nhóm endpoint hỗ trợ dashboard dành cho người bán.

Bảng A.4: Đặc tả API của Shop Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/api/v1/shops	Truy xuất danh sách toàn bộ cửa hàng đang hoạt động trên hệ thống.
GET	/api/v1/shops/{shop_id}	Lấy thông tin chi tiết cửa hàng theo mã định danh shop_id.
GET	/api/v1/shops/internal/{shop_id}	API nội bộ phục vụ tra cứu nhanh thông tin cửa hàng theo shop_id.
GET	/api/v1/shops/internal/by-user/{user_id}	API nội bộ phục vụ truy xuất cửa hàng dựa trên mã người dùng user_id.
GET	/api/v1/shop/dashboard/get_task_overview	Giao diện Seller: Lấy dữ liệu tổng quan và thống kê khối lượng công việc.
GET	/api/v1/shop/profile/get	Giao diện Seller: Lấy thông tin cấu hình hồ sơ cửa hàng.
POST	/api/v1/shop/profile/update	Giao diện Seller: Cập nhật thông tin hồ sơ cửa hàng.
GET	/api/v1/shop/product/list	Giao diện Seller: Truy xuất danh sách sản phẩm thuộc quyền quản lý của cửa hàng.

POST	/api/v1/shop/product/add	Giao diện Seller: Thêm sản phẩm mới vào danh mục kinh doanh.
POST	/api/v1/shop/product/update	Giao diện Seller: Cập nhật thông tin sản phẩm hiện có.

A.5. Product Service

Đặc tả cấu trúc định tuyến của phân hệ sản phẩm được tổ chức theo các nhóm router chính phục vụ quản lý dữ liệu hàng hóa.

Bảng A.5: Đặc tả API của Product Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/api/v1/products	Truy xuất danh sách sản phẩm trên hệ thống.
GET	/api/v1/products/{product_id}	Lấy thông tin chi tiết của sản phẩm theo mã định danh product_id.
POST	/api/v1/products	Tạo mới bản ghi sản phẩm.
PUT	/api/v1/products/{product_id}	Cập nhật thông tin của sản phẩm đã tồn tại.
DELETE	/api/v1/products/{product_id}	Xóa hoặc ẩn sản phẩm khỏi hệ thống.
GET	/api/v1/product/{product_id}	Tra cứu chi tiết sản phẩm theo luồng dữ liệu legacy.
POST	/api/v1/product/select_variation	Thực hiện lựa chọn biến thể sản phẩm theo luồng dữ liệu legacy.
GET	/api/v1/homepage/get-items	Truy xuất danh sách mặt hàng phục vụ hiển thị trên trang chủ.

GET	/api/v1/homepage/get_sections	Lấy dữ liệu cấu hình các phân mục hiển thị trên trang chủ.
GET	/api/v1/homepage/search	Thực hiện chức năng tìm kiếm sản phẩm từ giao diện trang chủ.

A.6. Order Service

Phân hệ quản lý toàn bộ vòng đời đơn hàng, hoạt động giỏ hàng và quy trình thanh toán thông qua các router chuyên biệt.

Bảng A.6: Đặc tả API của Order Service

Thành phần Router	Tiền tố cấu hình Prefix Router	Ghi chú vai trò phân hệ
Router chính	/api/v1/orders	Xử lý và cập nhật các nghiệp vụ liên quan đến đơn hàng.
Cart Router	/api/v1/cart	Thực hiện các thao tác thêm, xóa và cập nhật tài nguyên giỏ hàng.
Checkout Router	/api/v1/checkout	Quản lý quy trình kiểm tra thông tin, tính phí và khởi tạo đặt hàng.

A.7. Payment Service

Phân hệ trung gian xử lý giao dịch thanh toán và giao tiếp trực tiếp với cổng thanh toán điện tử VNPay.

Bảng A.7: Đặc tả API của Payment Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/api/v1/payment/payment_url?orderId=...	Tiếp nhận mã đơn hàng và tạo URL chuyển hướng thanh toán thông qua VNPay.
GET	/api/v1/payment/vnpay/return	Tiếp nhận và xử lý dữ liệu phản hồi callback từ VNPay.
POST	/api/v1/payment/order/{order_id}/pay	Khởi tạo bản ghi giao dịch thanh toán gắn với mã đơn hàng order_id.

A.8. Chat Service

Phân hệ quản lý cấu trúc hội thoại và hoạt động trao đổi tin nhắn giữa người mua và người bán.

Bảng A.8: Đặc tả API của Chat Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
GET	/api/v1/get_unread_rooms	Truy xuất danh sách phòng chat chứa tin nhắn chưa đọc.
GET	/api/v1/get_chatroom	Tra cứu thông tin chi tiết của một phòng chat cụ thể.
GET	/api/v1/get_chatroom_list	Lấy danh sách toàn bộ phòng hội thoại của người dùng.
GET	/api/v1/get_messages	Truy xuất lịch sử tin nhắn trong phòng chat.

POST	/api/v1/create_chatroom	Khởi tạo phòng hội thoại mới giữa các người dùng.
POST	/api/v1/messages	Gửi tin nhắn mới với đầu ra chuẩn hóa theo cấu trúc MessageResponse.
POST	/internal/send_message	Endpoint nội bộ phục vụ điều phối tin nhắn giữa các dịch vụ.

A.9. Upload Service

Dịch vụ đảm nhận tiếp nhận và lưu trữ dữ liệu đa phương tiện từ phía máy khách lên hệ thống lưu trữ đám mây.

Bảng A.9: Đặc tả API của Upload Service

Phương thức	Đường dẫn Endpoint	Chức năng nghiệp vụ / Ghi chú
POST	/api/v1/upload/image	Đăng tải hình ảnh lên hệ thống lưu trữ đám mây và trả về cấu trúc UploadResponse.
POST	/api/v1/upload/video	Đăng tải video lên hệ thống lưu trữ đám mây và trả về cấu trúc UploadResponse.
POST	/api/v1/upload/file	Đăng tải tệp tài liệu hoặc tệp dữ liệu chung lên hệ thống lưu trữ đám mây và trả về cấu trúc UploadResponse.
DELETE	/api/v1/upload/{bucket}/{object_name:path}	Xóa vĩnh viễn tệp lưu trữ dựa trên bucket và object_name.
GET	/api/v1/upload/signed-url/{object_name:path}	Khởi tạo signed URL có thời hạn truy cập dựa trên object_name.